Multi-Tenant POS Backend — 2-Hour Fast-Track Interview Sheet

The High-Impact Summary: Core Story, Architectural Decisions, Code Highlights & Q&A; Prep

1. Your Spoken Elevator Pitch (Say This First)

"Tell me about your project" — Memorize This 60-Second Response:

"I engineered a high-concurrency, multi-tenant Point of Sale backend in NestJS, PostgreSQL, and Redis. It solves the 3 biggest problems in retail software: **inventory overselling during flash sales**, **duplicate payments on network drops**, and **cross-tenant data leakage**.

To guarantee zero overselling, I implemented row-level pessimistic locking (**SELECT FOR UPDATE**) in TypeORM with **ascending product ID sorting** to mathematically prevent deadlocks, backed by a database **CHECK (quantity >= 0)** constraint.
To prevent double charges, I created a distributed **Idempotency Interceptor** that hashes the payload with SHA-256 and replays cached receipts on retries.
To prevent connection pool exhaustion, I used a **Two-Phase SAGA**: stock is decremented in milliseconds, locks are released, and payment is processed outside the database transaction with an automated compensating rollback if payment fails.
Catalog reads and tenant checks are cached in **Redis** with write-through invalidation, and the whole system is covered by automated Jest concurrency and e2e tests."

2. The Story Behind the Project (Explain the Problem)

The Problem: Ramesh owns a specialty coffee chain called *'Chai & Roast'* with stores in Bengaluru and Mumbai. With off-the-shelf POS software, he had 3 recurring nightmares: (1) **Ghost Stock:** When 1 bag of beans remained, two cashiers rang it up simultaneously and stock dropped to -1. (2) **Double Swipe:** Airport store Wi-Fi lagged, cashier hit 'Pay' twice, charging customer twice. (3) **Data Leaks:** Store managers could see competitor tenant sales numbers.
The Solution: This backend provides mathematical guarantees: inventory can never oversell, retries never double charge, tenants are strictly isolated, and money is tracked in 100% exact integer minor units (paise).

3. Roles & Permissions Hierarchy (RBAC)

Our system uses a **numeric hierarchy** evaluated in src/common/guards/roles.guard.ts: OWNER (4) > ADMIN (3) > MANAGER (2) > CASHIER (1).

Role	Level	Scope	What They Can Do	What They CANNOT Do
OWNER	4	Merchant	Full corporate control: Update merchant info, deactivate stores, create Admins/Managers/Cashiers, view all store sales.	Cannot access Tenant B data.
ADMIN	3	Merchant	Manage operations: Create stores, create users (POST /users), create/edit products, manage catalog.	Cannot deactivate stores or change corporate details.
MANAGER	2	Store	Store supervisor: View assigned store, set stock levels (PUT /inventory/:pId), view cashiers and sales.	Cannot create stores, create users, or edit other stores.
CASHIER	1	Store	POS counter operator: Browse products, check stock, process sales (POST /sales), print receipts.	Cannot adjust stock directly, create users, or access other stores.

Key Rule: OWNER & ADMIN do not need a store ID. MANAGER & CASHIER *must* be assigned to a specific store ID.

4. The 3 Technical Highlights (Interview Winning Points)

Highlight A: Pessimistic Row Locking & Deadlock Prevention

In src/modules/inventory/inventory.repository.ts, we execute setLock('pessimistic_write') which generates SQL SELECT ... FOR UPDATE. PostgreSQL locks the specific inventory rows until the transaction commits.

```
// Deadlock Prevention in SalesService:
const sortedProductIds = [...productIds].sort(); // 1. Always sort IDs ascending!
const lockedInventory = await this.inventoryRepo.lockAndFetchForUpdate(em, storeId, sortedProductIds);
// Why sort? If Tx1 locks (A then B) while Tx2 locks (B then A), a circular deadlock happens.
// Sorting guarantees ALL transactions lock resources in identical order -> ZERO DEADLOCKS!
```

Highlight B: Two-Phase SAGA with Compensation (Payment Outside DB Lock)

Senior Dev Insight: Never make third-party HTTP calls (Stripe/Razorpay) inside an open database transaction. If the gateway takes 4 seconds, database locks stay open and exhaust your connection pool.

```
// Two-Phase SAGA in SalesService:
// PHASE 1: DB Transaction (< 5ms) -> Lock stock, validate qty, decrement stock, COMMIT & release locks!
// PHASE 2: External HTTP -> Call paymentPort.charge() OUTSIDE the transaction (zero DB locks held).
// OUTCOME:
// - If Payment SUCCESS -> DB Transaction 2A marks sale PAID and records payment.
// - If Payment FAILED  -> DB Transaction 2B executes COMPENSATION: increments stock back to original!
```

Highlight C: Distributed Idempotency Interceptor (Network Retry Safety)

In src/common/interceptors/idempotency.interceptor.ts: Clients send an Idempotency-Key header (UUID). The interceptor hashes method + path + JSON body using SHA-256.

```
// Replay Logic:
if (existingKey.isCompleted) {
  if (existingKey.requestHash !== currentHash) throw new UnprocessableEntityException('Key reused with different payload');
  res.setHeader('X-Idempotency-Replayed', 'true'); // Instant replay from DB cache!
  return of(existingKey.responseBody);
}
```

5. Database Design & Integrity (9 Tables)

1. **Absolute Zero-Oversell Guarantee:** CONSTRAINT "CHK_store_inventory_quantity" CHECK ("quantity" >= 0) in Postgres. Physical guarantee.
2. **Integer Minor Units (BigInt):** All money is stored in paise (■450.00 = 45000). Eliminates IEEE 754 floating-point rounding errors.
3. **Immutable Price Snapshots:** sale_items stores unitPriceSnapshot so future catalog price changes never alter past receipts.
4. **Core Tables:** merchants, stores, users, products, store_inventory, sales, sale_items, payments, idempotency_keys.

6. Top 10 Toughest Interview Questions & Exact Spoken Answers

Q1: How do you prevent inventory overselling during concurrent purchases?

Answer: We use row-level pessimistic locking (SELECT FOR UPDATE) inside an atomic transaction. When Transaction 1 queries the stock row, PostgreSQL places an exclusive lock on it. If Transaction 2 arrives concurrently, it must wait until Transaction 1 commits. Transaction 1 decrements stock and commits. Transaction 2 then inspects the updated row, sees insufficient stock, and throws a 422 Unprocessable Entity error. Our database also enforces CHECK (quantity >= 0) as a hard constraint.

Q2: How do you prevent deadlocks when locking multiple inventory rows?

Answer: Deadlocks occur when concurrent transactions acquire locks in opposing orders. We prevent this by sorting product IDs in ascending order ([...productIds].sort()) before calling lockAndFetchForUpdate. Because every transaction acquires locks in the exact same sequence, circular wait deadlocks are mathematically impossible.

Q3: Why is payment processed outside the database transaction?

Answer: Database transactions hold row locks and consume connection pool slots. If an external payment gateway takes 3-5 seconds due to network latency, keeping the transaction open would freeze inventory rows and exhaust the connection pool. We use a Two-Phase SAGA: stock is decremented in milliseconds, the lock is released, payment is called outside, and if it fails, a compensating transaction increments the stock back.

Q4: How does your Idempotency mechanism work?

Answer: The client passes an Idempotency-Key header. Our IdempotencyInterceptor hashes the method, path, and JSON body with SHA-256. If a request arrives with an existing key, we verify the hash. If the hashes match, we immediately replay the stored response with X-Idempotency-Replayed: true, preventing duplicate orders or payments. If the payload differs, we throw a 422 error.

Q5: How do you enforce Multi-Tenancy and prevent cross-tenant data leaks?

Answer: Multi-tenancy is enforced at three levels: (1) The JWT payload securely carries user.merchantId and user.storeId. (2) StoreAccessGuard validates that the requested store belongs to the user's merchantId and managers/cashiers only access their assigned store. (3) Database queries filter by merchantId and storeId.

Q6: Why store prices as integers instead of decimals or floats?

Answer: Floating-point arithmetic introduces IEEE 754 precision errors (e.g. 0.1 + 0.2 = 0.30000000000000004). Over thousands of transactions, this causes accounting discrepancies. We store currency in integer minor units (paise/cents, where ■450.00 = 45000 in a bigint column), guaranteeing exact financial math.

Q7: How does your Redis caching strategy work?

Answer: We cache product catalog reads (GET /products) in Redis for 5 minutes (TTL = 300,000ms). When an Admin creates, updates, or deletes a product, ProductsService explicitly invalidates the merchant's catalog cache key (products:merchant:). We also cache store existence in StoreAccessGuard for 60 seconds to eliminate redundant database queries.

Q8: How are passwords secured?

Answer: Passwords travel over TLS/HTTPS and are hashed using bcrypt with a salt cost factor of 10 (1,024 rounds). Passwords are never stored in plain text, never logged, and the passwordHash property is explicitly stripped before returning user entities.

Q9: What external integrations exist vs what is NOT implemented?

Answer: We implemented an abstract PaymentPort with a MockPaymentProvider for testing success and failure scenarios, and a daily 2:00 AM cron job (@nestjs/schedule) to clean up expired idempotency keys. Email (Nodemailer) and Message Queues (Kafka/BullMQ) are NOT in the codebase; they are part of our future roadmap.

Q10: What would you improve in Version 2.0?

Answer: (1) Integrate BullMQ or Kafka for asynchronous receipt emails and analytics event streaming. (2) Implement a Redis token blacklist for immediate revocation on user logout. (3) Add Helmet and Throttler modules for edge rate limiting.

7. Quick 7-Step Postman Demo Guide

If the interviewer asks you to demo via Postman or explain your testing workflow, follow these 7 steps:

1. **Health Check:** GET /api/v1/health -> Returns { status: 'ok', db: 'ok', redis: 'ok' }.
2. **Owner Login:** POST /api/v1/auth/login with owner-a@merchant.com / password123. Copy the returned accessToken.
3. **Create Store:** POST /api/v1/stores with Bearer token. Body: { "name": "Downtown Flagship", "address": "Indiranagar" }.
4. **Create Cashier:** POST /api/v1/users. Body: { "email": "cashier@store.com", "password": "password123", "role": "CASHIER", "storeId": "<STORE_ID>" }.
5. **Set Stock:** PUT /api/v1/stores/:id/inventory/:productId with Manager token. Body: { "quantity": 10 }.
6. **Process Sale:** POST /api/v1/stores/:id/sales with Cashier token & Idempotency-Key: test-uuid-1. Status: **201 Created**.
7. **Prove Idempotency:** Send the exact same request again. Header shows X-Idempotency-Replayed: true and stock is NOT decremented twice!

8. 60-Second Last-Minute Revision Checklist

Read This Right Before You Join the Interview Call:

- **Roles:** 4 roles. OWNER (4) > ADMIN (3) > MANAGER (2) > CASHIER (1). Evaluated numerically in RolesGuard.
- **Concurrency:** SELECT FOR UPDATE + sorted product IDs (deadlock prevention) + CHECK (quantity >= 0).
- **SAGA:** Fast DB lock release (< 5ms) -> payment outside -> compensating rollback if payment fails.
- **Idempotency:** SHA-256 payload hash + replay cached response with X-Idempotency-Replayed header.
- **Caching:** Redis caches product catalog (5 min) with write-through invalidation; store access cached 60s.
- **Truth on Features:** PaymentPort is a mock adapter; Email and Kafka/BullMQ are NOT in code (planned for v2).

Good Luck!

Take a deep breath. You built an exceptionally clean, well-architected project. Be calm, speak clearly, walk through the story of Ramesh and your concurrency decisions, and you will do great!